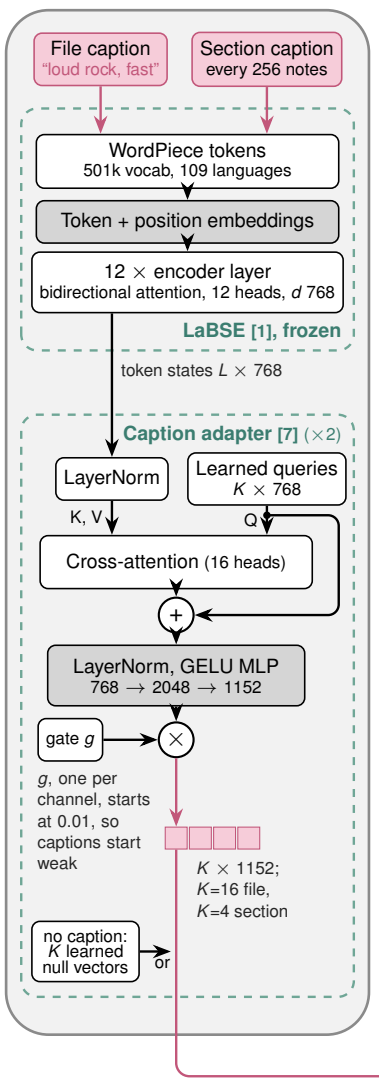
Captioned music model (477M)

477M trainable parameters: trunk 446M, cascade heads 16M, two caption adapters 13M, embeddings 2M. The LaBSE text encoder (471M) is frozen and not counted.

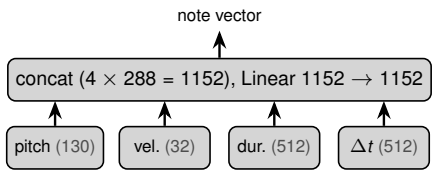
Status: the music model without caption adapters (462M) is trained. The caption stage of this layout (F1) is planned; caption results here come from 107M pilots.

How to read: numbers give the reading order; pink = captions; grey boxes = linear layers and embedding tables; dashed frame = a module drawn open in place; dotted frame or box = a zoom (white panel) into the dotted part.

1 Caption path



2 Input embedding

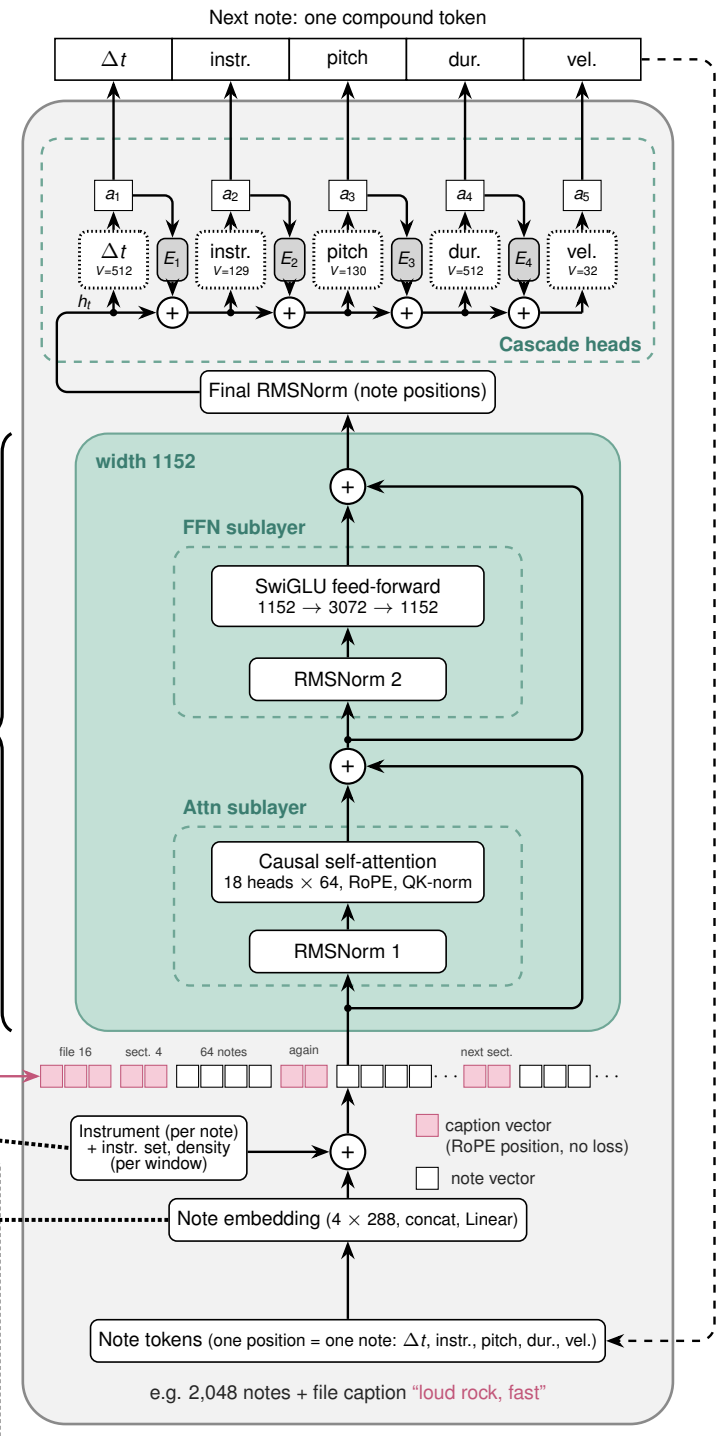


+ instrument of this note: a table of 129 vectors (128 General MIDI programs + drums), added to the note vector. The output head *instr.* predicts the next one.

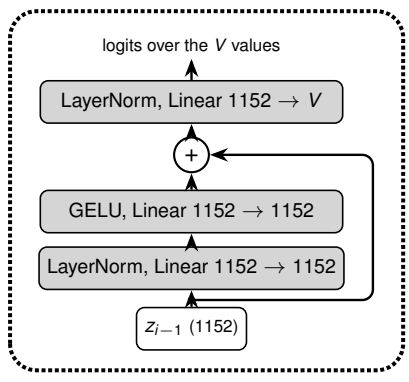
+ window conditions, the same vector at every position: the **instrument set** of the window (129-dim multi-hot → Linear) and its **density** (notes per second in 15 levels + "none" → a table of 16 vectors). Both start at zero and are dropped 15% of the time in training, so "none" stays usable. In generation they come from the user or are measured on the prompt.

Time is absolute: Δt and durations in 20 ms steps, clipped at 10.22 s. There are no bars, beats or tempo tokens; the metre has to be read from the timing. 2,048 notes are about 1–4 minutes of music.

3 Music model



4 One cascade head (the 5 dotted boxes, own weights)



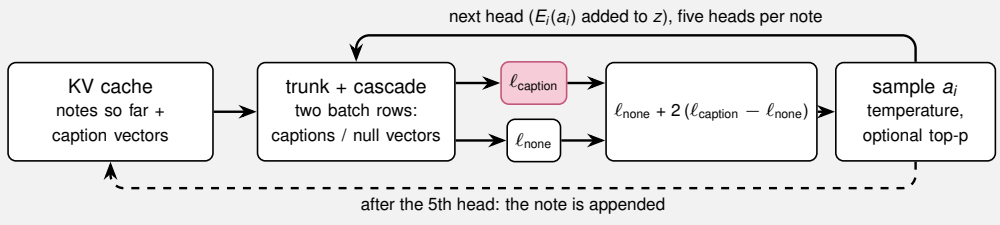
Why a cascade. One position predicts all five values of the next note. Head i reads the running state

$$z_0 = h_t, \quad z_i = z_{i-1} + E_i(a_i),$$

the trunk state h_t plus the embeddings E of the values already chosen. So the note's probability follows the chain rule, $p(\Delta t)\, p(\text{instr.} \mid \Delta t)\, p(\text{pitch} \mid \Delta t, \text{instr.}) \cdots$ (compound tokens as in [6]).

a_i is the true value in training (teacher forcing) and the sampled value in generation. Loss: the sum of the five cross-entropies.

5 Generation



Classifier-free guidance [9] with scale 2: every step runs the batch twice as wide, once with the captions and once with null vectors.

Caption vectors enter the cache as in training: the file caption first, then a section's 4 vectors at its first note and again every 64 notes. Past 2,048 notes the window slides by a quarter.

6 Training recipe for section captions

Without these measures the model follows the first section's caption and then ignores later ones.

1. Change-aware captions. Each section caption names what changed since the section before ("drums enter, louder"). Half of the training windows contain a section change.

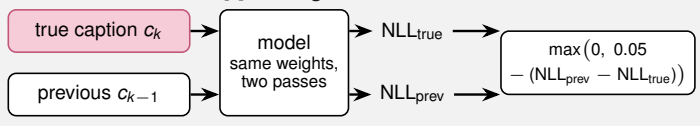
2. Spliced windows (25% of windows).

File A up to a section boundary, then a section of file B (dashed) with B's caption and no file caption: the caption contradicts the context and has to win.

3. Caption repeated every 64 notes, in training and in generation.

The same 4 section vectors are inserted again, so the caption is never far back in the context.

4. Counterfactual loss [8] + recognition head.



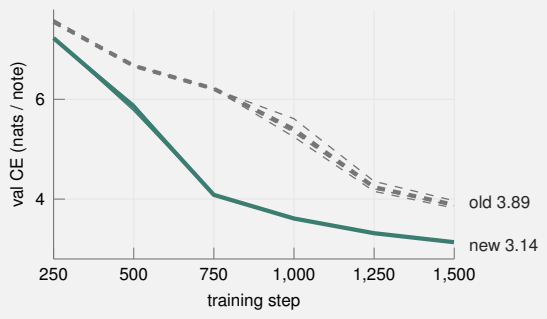
NLL is measured on the first 32 notes of a changed section. The true caption must beat the previous one by 0.05 nats (weight 1.0). A linear head on those notes' hidden states tells the true caption from the previous one (BCE, weight 0.1). Both are used in training only.

5. Caption dropout → guidance. File captions are replaced by null vectors 15% of the time, section captions 20%, so generation can use classifier-free guidance [9] with scale 2.

Evidence so far (107M pilot, 1,500 steps, one seed). Switch test: 6 prompt pairs A/B (e.g. soft / loud), 16 samples each, the section caption changes at note 1,024, guidance 2. The matching attribute (velocity, notes per second, register, drum or string share) is measured per 256 notes, and the score is the share of a full flip (0 = keeps the first style, 1 = follows the new caption). Before measures 2–4 (v2_A0): **0.01**; with all three: **0.85**; splice alone 0.59, counterfactual alone 0.13, repeat alone 0.09. Held-out NLL 2.783 vs 2.772 for the same steps without them. No error bars yet.

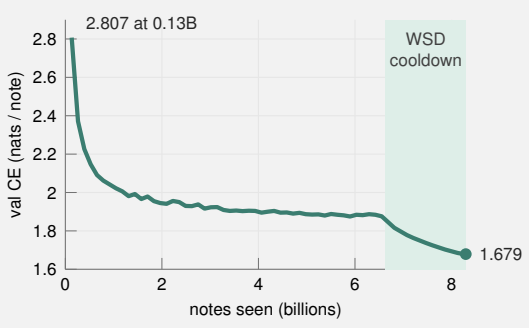
7 Why these choices: training curves (teal = what the model uses)

Trunk block (42M, 98M notes, 3 seeds each)



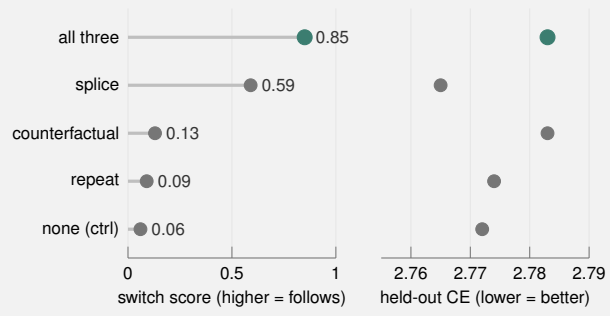
Clean GigaMIDI validation. **New**: RMSNorm, SwiGLU, QK-norm; **old** (dashed): LayerNorm, GELU; both RoPE and Muon. Thin lines = seeds, thick = mean; −0.75 at the end.

Base model pretraining (462M, 8.3B notes)



Clean GigaMIDI validation every 2,000 steps (65,536 notes per step); constant LR, then the $1 - \sqrt{\cdot}$ cooldown in the last 20%. One run on one RTX 5090, ≈41 h.

Section-caption measures (107M pilot, 1,500 steps)

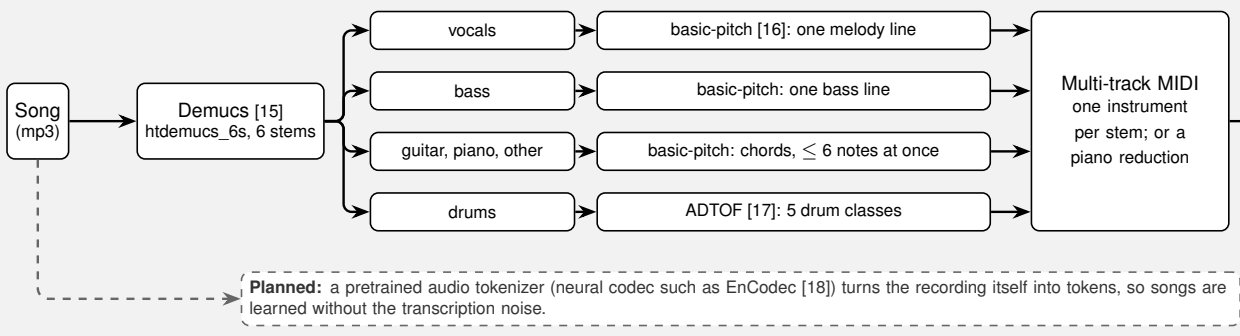


Each row continues the same model (v2_A0) for 1,500 steps with one measure, or all three. Switch score as in section 6; one seed, no error bars. **Teal** = the recipe in use.

8 Training stages

Setting	Base pretraining (done)	Caption training, F1 run (planned)	Fine-tune on Ukrainian songs
Start	from scratch	the base + the two new caption adapters	the base (done, no captions); the F1 model with captions (F2, planned)
Data	GigaMIDI [12] 0.4, Aria-MIDI [13] 0.3, Discover MIDI [19] 0.3; 8.3B notes (1.8 epochs)	captioned MIDI in 3 tiers: MidiCaps [14] / XMIDI [20] / GigaMIDI metadata, Aria-MIDI metadata, the rest with feature captions; 2.5–4B notes	5,358 songs transcribed from audio (section 9) 0.75 + GigaMIDI replay 0.25 against forgetting
Captions	none: only the instrument-set and density conditions	templates filled from metadata (genre, mood, artist, composer) and from features computed from the notes (tempo, loudness, register, instruments), 30% in Ukrainian; MidiCaps' LLM-written captions; section captions from each section's own notes	none in the done run; F2 adds them
Window	2,048 notes (≈1–4 min)	2,048 notes + 16 file vectors + 4 per caption arrival (section start and every 64 notes): ≈2,190 positions	2,048 notes
Batch	65,536 notes per step (32 windows)	same	24,576 notes per step (12 windows)
Optimizer, LR	Muon [10] for weight matrices, AdamW for the rest; one LR 10^{-3} (Muon updates scaled to AdamW's size)	AdamW, LR 10^{-4}, adapters 10× (as in the 107M pilots)	8-bit AdamW, LR $2 \cdot 10^{-4}$
Schedule	WSD [11]: warmup, constant, $1 - \sqrt{\cdot}$ cooldown over the last 20%	same	same
Steps	126,582 (≈41 h on one RTX 5090)	≈38–61k	1,500
Precision	bf16 with fp8 matmuls, torch.compile	same	bf16

9 Audio → MIDI: how songs become training notes (fine-tune data)



Every step adds errors: stem bleed, missed or extra notes, octave and timing errors, drum classes mixed up, loudness guessed from activations; stems quieter than −30 dB vs the mix are dropped. The model learns these errors as if they were music.

10 Glossary

Δt	time since the previous note started (20 ms steps, 512 values)	**RoPE [2]**	rotary position embedding: positions as rotations of queries and keys
instr.	instrument: General MIDI program 0–127 or drums (129 values)	**RMSNorm [3]**	layer normalisation by the root mean square only
pitch	MIDI pitch 0–127 + start / end of piece (130 values)	**SwiGLU [4]**	feed-forward layer with a SiLU-gated linear unit
dur.	note length (20 ms steps, 512 values)	**QK-norm [5]**	queries and keys normalised before the dot product
vel.	velocity: how hard the note is played (32 levels)	**LaBSE [1]**	multilingual sentence encoder (109 languages)
compound token	one sequence position = one whole note	**NLL**	negative log-likelihood of the true notes
caption vector	a caption-adapter output placed in the sequence; no loss on it	**nats**	units of NLL with the natural log
null vectors	learned stand-ins for "no caption"	**BCE**	binary cross-entropy (two-class loss)
teacher forcing	training feeds the true earlier values, not samples	**CFG [9]**	classifier-free guidance: push logits away from the caption-free prediction
KV cache	keys and values of earlier positions kept between steps	**WSD [11]**	warmup, stable, decay: LR schedule with a short final cooldown
multi-hot	a 0/1 vector with a 1 for every instrument present	**Muon [10]**	optimizer that orthogonalises the update of each weight matrix
stem	one separated part of a recording (vocals, bass, drums, ...)	**Demucs [15]**	neural source separation: a song into 6 stems

11 References (links)

[1] LaBSE: Language-agnostic BERT Sentence Embedding (2020), arXiv:2007.01852
[2] RoFormer: rotary position embedding (2021), arXiv:2104.09864
[3] Root Mean Square Layer Normalization (2019), arXiv:1910.07467
[4] GLU Variants Improve Transformer (2020), arXiv:2002.05202
[5] Query-Key Normalization for Transformers (2020), arXiv:2010.04245
[6] Compound Word Transformer (2021), arXiv:2101.02402
[7] Flamingo: learned-query resampler, gated adapters (2022), arXiv:2204.14198
[8] MuseBarControl: counterfactual loss (2024), arXiv:2407.04331
[9] Classifier-Free Diffusion Guidance (2022), arXiv:2207.12598
[10] Muon is Scalable for LLM Training (2025), arXiv:2502.16982
[11] Scaling laws beyond fixed training durations: WSD (2024), arXiv:2405.18392
[12] The GigaMIDI Dataset (2025), arXiv:2502.17726
[13] Aria-MIDI: a dataset of piano MIDI files (2025), arXiv:2504.15071
[14] MidiCaps: MIDI with text captions (2024), arXiv:2406.02255
[15] Demucs: Hybrid Transformers for Music Source Separation (2022), arXiv:2211.08553
[16] basic-pitch: lightweight polyphonic note transcription (2022), arXiv:2203.09893
[17] ADTOF: automatic drum transcription dataset and model (2021), arXiv:2111.11737
[18] EnCodec: High Fidelity Neural Audio Compression (2022), arXiv:2210.13438
[19] Discover MIDI Dataset, Hugging Face dataset
[20] XMusic: the XMIDI dataset (2025), arXiv:2501.08809